

The end of Software Engineering?

How AI agents are changing the way softwares are designed, engineered and delivered.

29 Sep, 2026

Industry Connect Seminar - Dept. of Mathematics, CEG, Anna University - 29 Sep, 2026

Vikram Singh - Founder & Product Architect - Zestifai Technologies

The SWE paradigm shift over 2023-26

2023 - 24

Human asks LLM, LLM explains/generates, human implements.

Prompt engineering becomes a software skill.

LLM decides which operations to perform using tool use.

Unit tests becomes insufficient for probabilistic systems -> evals

AI-assisted SDLC becomes mainstream

2025 - 26

Human describes objective, agent researches/plans/implements/tests, human reviews.

Agentic Coding -> Devs delegate tasks directly from terminal.

Agentic DevOps Loop -> "Issue -> Agent -> Branch -> Code -> Tests -> PR -> Human Review -> Merge -> Deploy"

Software specifications become machine-readable engineering artifacts

AI moves from answering >> generating >> executing

What has changed?

1. English Language is now a first-class programming interface for humans.
2. Understanding code is more important than writing code.
3. IDE centric workflow has given way to Agent centric workflow.
4. Developers now work on multiple agents/tasks in parallel.
5. Specification + Architecture + Judgement are now in-demand skills
6. Documentation more critical, part of the codebase, context for agents.
7. Deterministic testing gave way to dynamic, probabilistic evaluations (evals).
8. Context, models, tools, memory, evals, sandbox, Git, CI are the new dev tools

...

Addition to SWE vocabulary; assorted

Context, Context Rot

Vibe Coding, AI Slop

LLMs, Prompts, Agents

Graph Engineering

MCP, A2A, UCP..

Function/Tool Calling

Token anxiety

AGENTS.md/CLAUDE.md

Git + Worktrees

Agent Sandboxing

Tokens, Embeddings

Harness Engineering

Evals, LLM-as-a-judge

Kimi/GLM/Qwen..

Claude/OpenAI/Gemini..

Model Hallucination

RAG, Vector DBs

...

Unit of work got bigger

Function → Module → Service → Workflow → Agent Task

Function: A local change. Performs one focused operation within a component.

Ex. `calculateOrderTotal()` >> calculates the total for one order.

Module: A coherent component. Owns a related set of responsibilities.

Ex. Order Management >> handles order creation, pricing, status, and cancellation.

Service: A deployable boundary. Owns a capability independently of other components.

Ex. Order Service >> independently exposes APIs for creating and managing orders.

Workflow: A coordinated process. Orchestrates multiple steps, components, or services toward an outcome. Ex. Order Fulfilment >> payment → inventory → shipping → notification.

Agent Task: A goal. Owns an outcome and decides the steps, tools, and actions needed to achieve it. Ex. Resolve a failed order >> investigate failure, inspect payment/inventory/shipping state, take permitted actions, and report the resolution.

Unit of delegation is getting bigger

Token/Line Level: Ask AI for local completion

Function/File Level: Ask AI to implement a bounded change.

Feature/Task Level: Human describes an outcome and system determine a sequence of changes.

Workflow/Pipeline Level: Multiple steps, tools, checks and hand-offs can be coordinated by AI

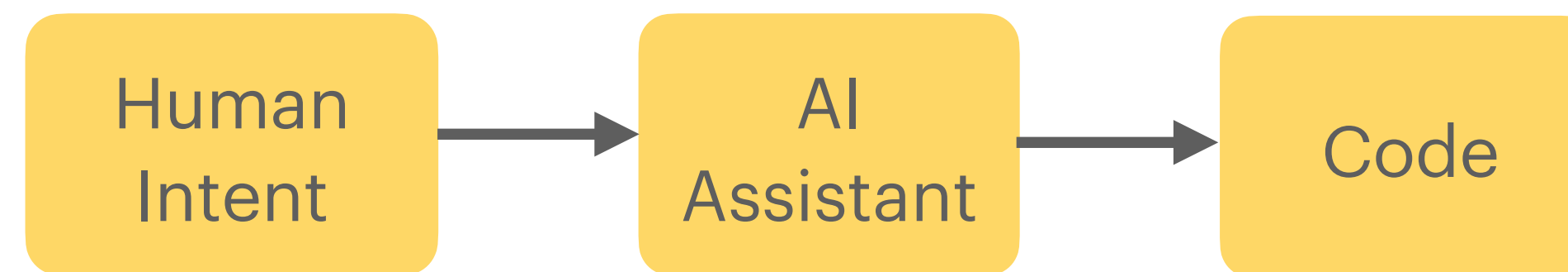
Token >> Line >> Function >> File >> Feature >> Task >> Workflow.

(Human role shifts)

Do >> Assist >> Delegate >> Supervise.

AI Assisted to Agentic Engineering

AI Assisted Coding: AI helps build the software and human remains in the critical path for architecture, integration, verification, and delivery



Agentic Engineering: The agent drives the engineering loop. Human shifts from author to intent architect + auditor/supervisor. Output shifts from software to outcome. Shift form writing software >> To engineering systems that produce software.



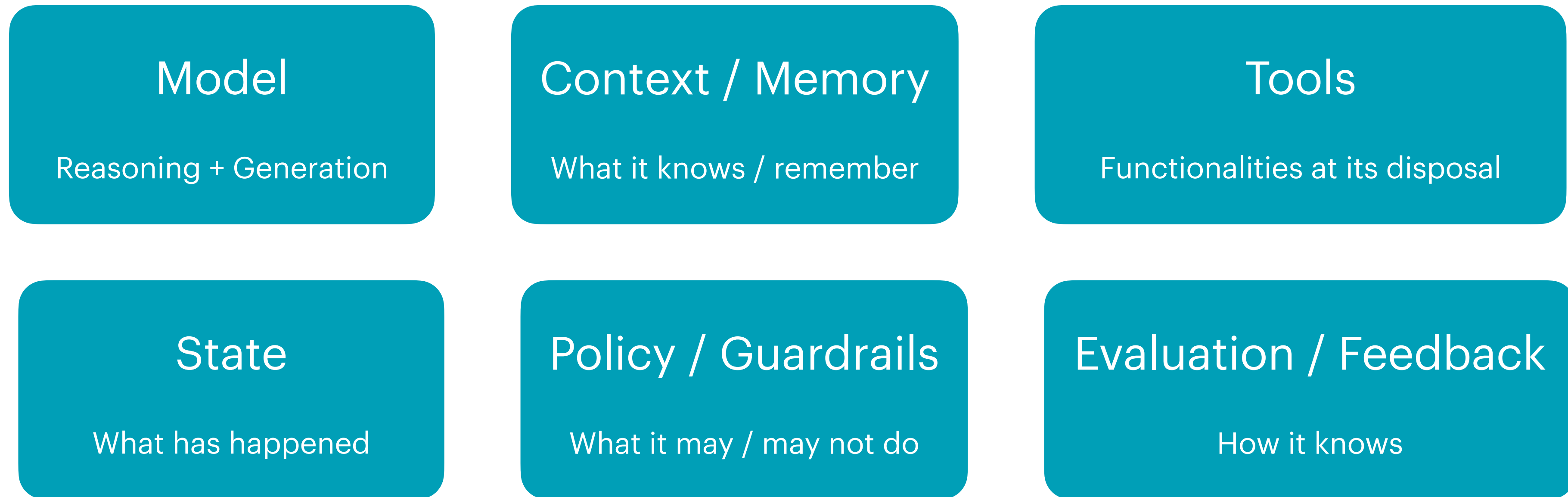
Adopted from the paper “The End of Software Engineering” by Zhenfeng Cao - <https://arxiv.org/html/2606.05608v1>

What are AI Agents?

An **AI Agent** is a software system that uses an AI model(s) to pursue a goal by deciding and executing a sequence of actions in an environment, using tools, state, context and feedback, until a defined outcome or stopping condition is reached.

AI Agent = Goal + Reasoning + Tools + State + Feedback + Autonomy

Key Components of AI Agents



Model $\neq$ Agent

Key Capabilities of AI Agents

Perceive

Read repo, docs, data

Reason

Analyse and Choose

Plan

Break Goal into Steps

Act

Edit, Run, Call tools

Observe

Inspect results

Adapt

Correct & Retry

Agents take over coding

The bottle moves up/down stream

Throughput

Agents raise the throughput of plausible code. I

Proof

The verification tax: Time saved writing is often re-spent auditing.

Scarcity

Scarce work moves upstream and downstream.

Intent · Specification · Architecture ← Coding → Tests · Evals · Observability · Judgment
(Upstream work) (Downstream work)

No more a scarce skill

<https://dora.dev/research/2025/dora-report/> | <https://dora.dev/insights/balancing-ai-tensions>

Engineers sits above the code; Skill bar moving up



Human Intent: What problem? Why? What constraints?

Specification: What exactly must be true?

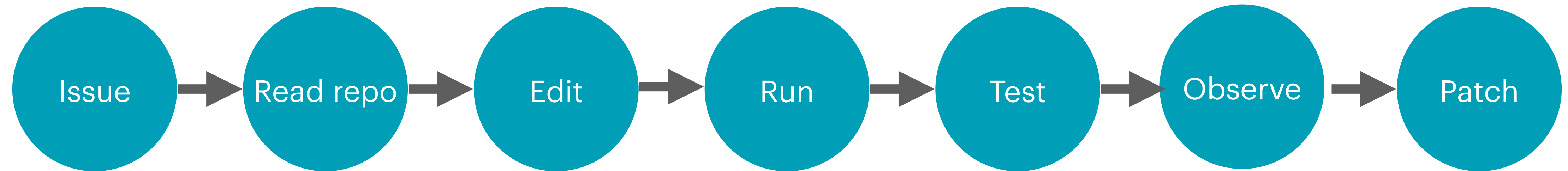
Context & Harness: What can the agent know and do?

Agent & Workflow: How will it act, observe and iterate?

Evidence: How do we know it is done?

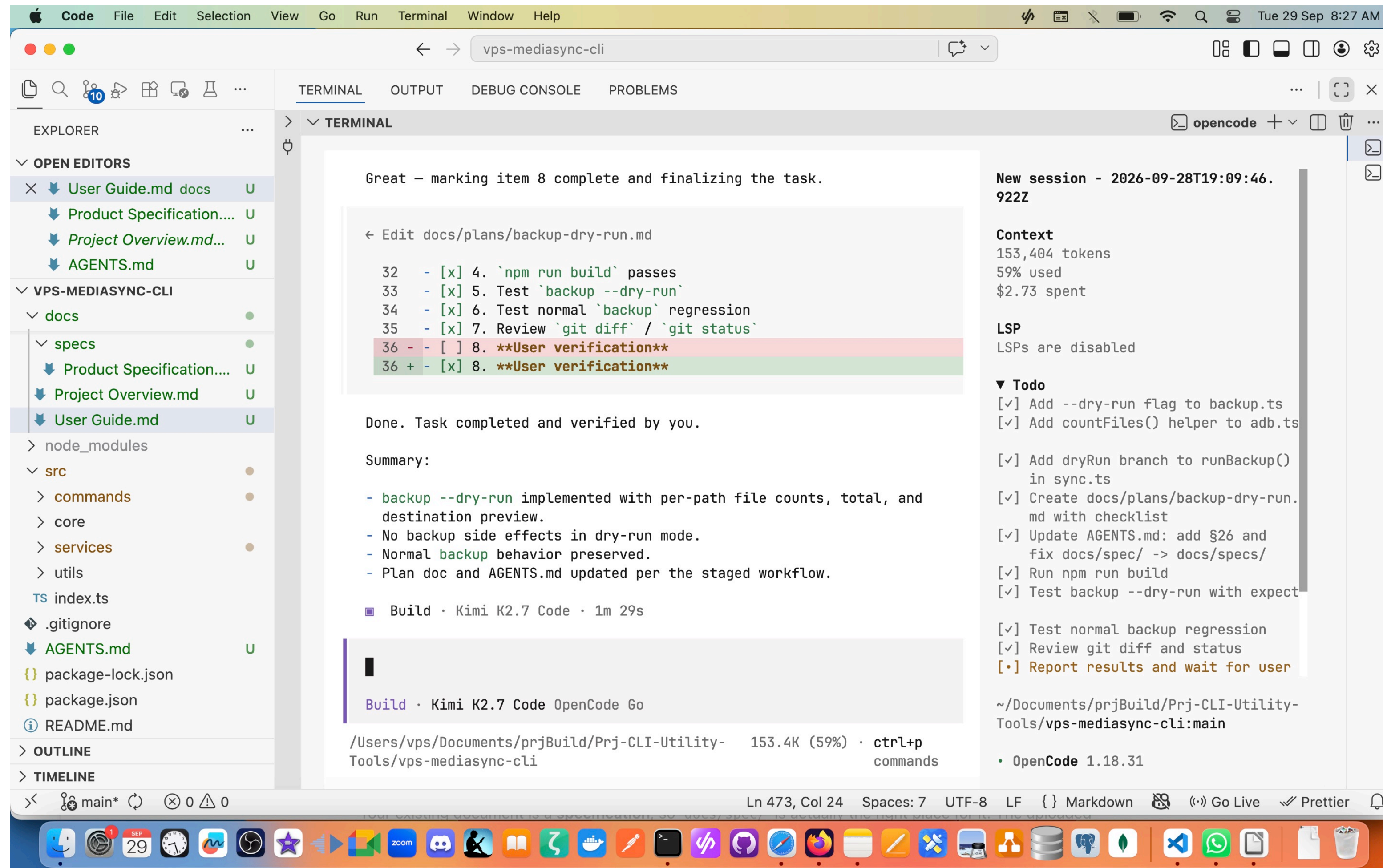
Software: The artifact is the output, not the whole engineering problem.

The coding agent does more than answer



- A chat model returns an answer.
- A coding agent operates inside an environment and can take multiple actions.
- The AI is now participating in the engineering loop.

Introducing OpenCode, A Coding Agent



Coding Agent Demo

New Software Engineering Stack

One Stack, Six Ideas

Reliable Software: Evidence + Recovery

Harness: What can I do?

Graph: What happens next?

Loop: Should I continue?

Agent: Choose and Act.

Model: Reason and Generate.

Model capability $\neq$ Agent capability $\neq$ System capability

Model

Writes SQL
Writes/Explains code
Suggests a plan
...

Agent

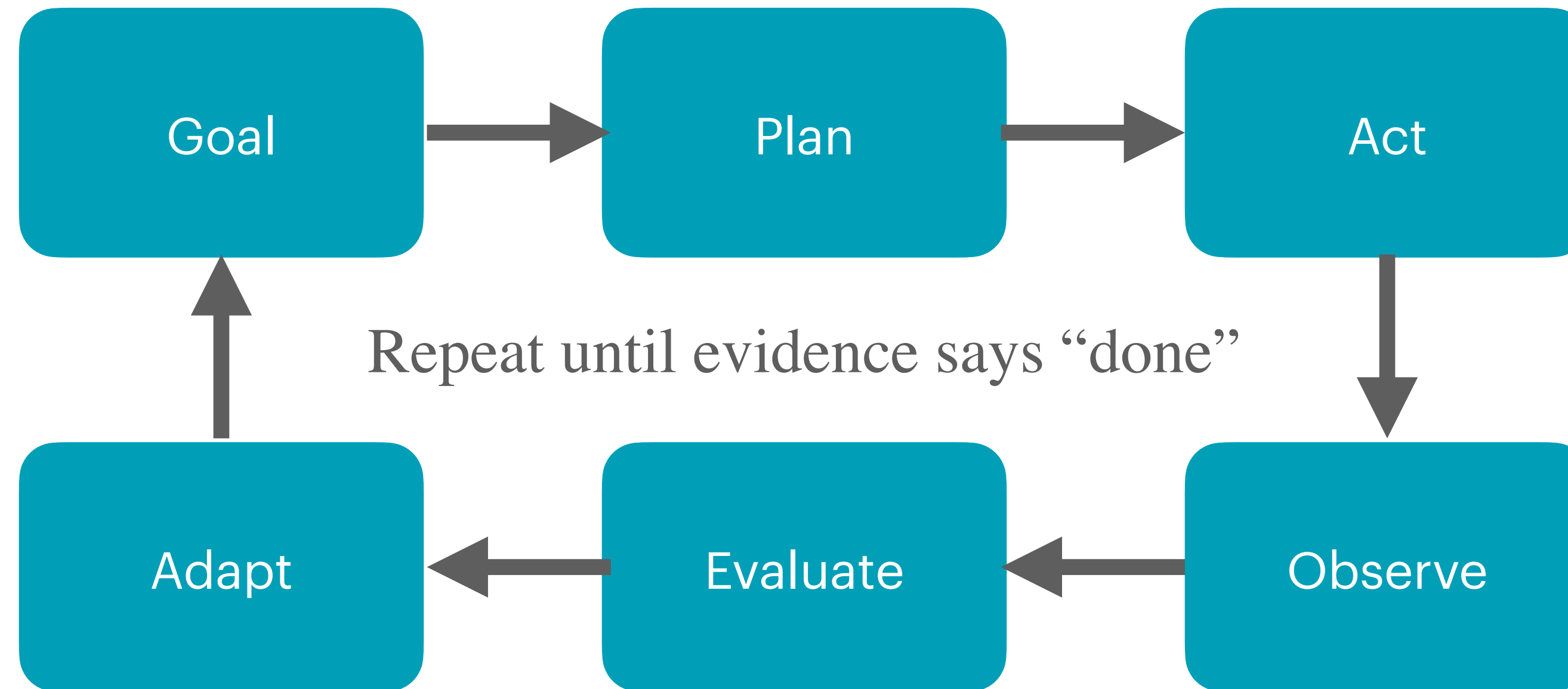
Reads schema
Calls tools
Runs tests
Iterates
...

System

Policy
Permissions
Evidence
Recovery
Audit
...

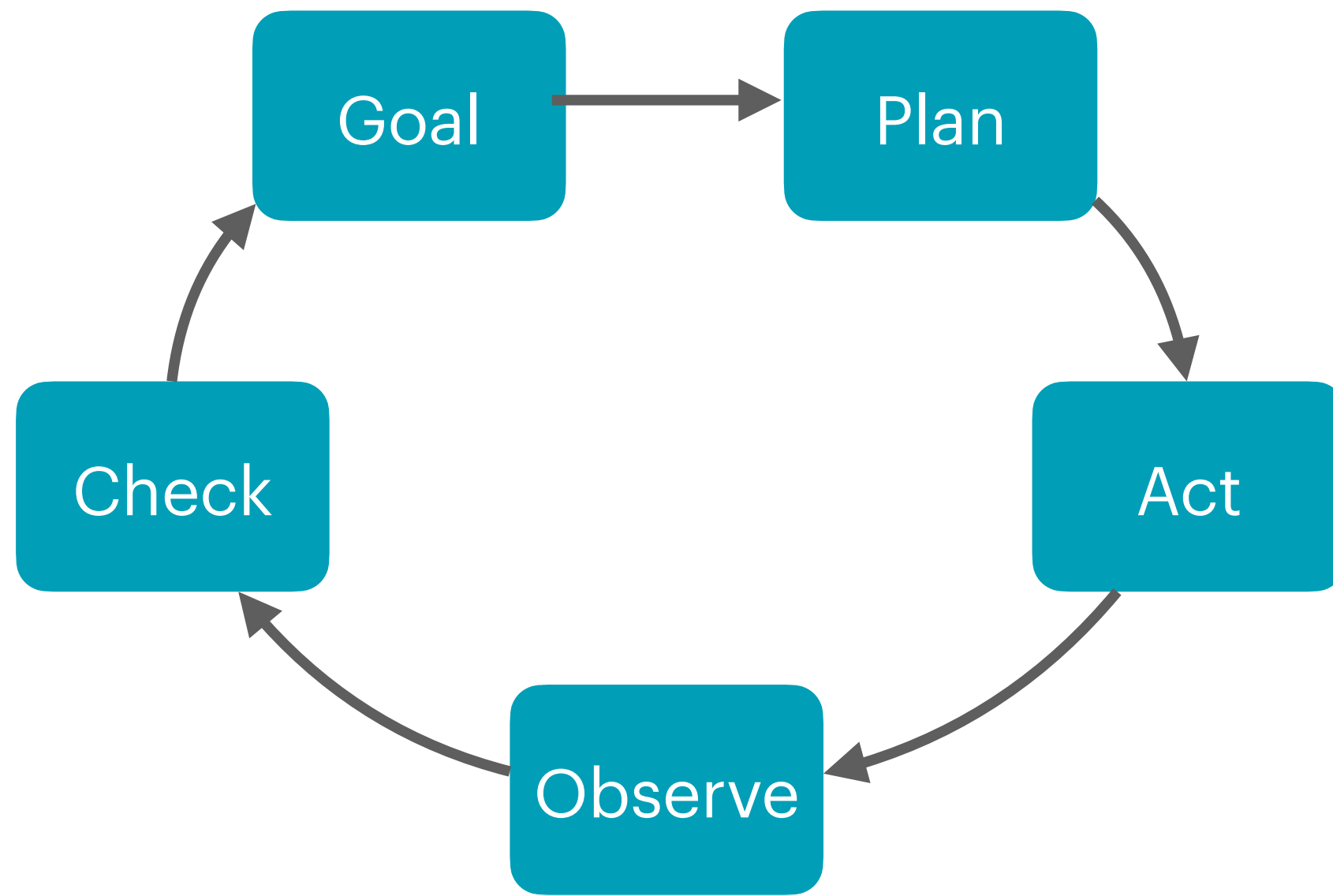
The engineering challenge moves outward from the model.

The Agent Loop

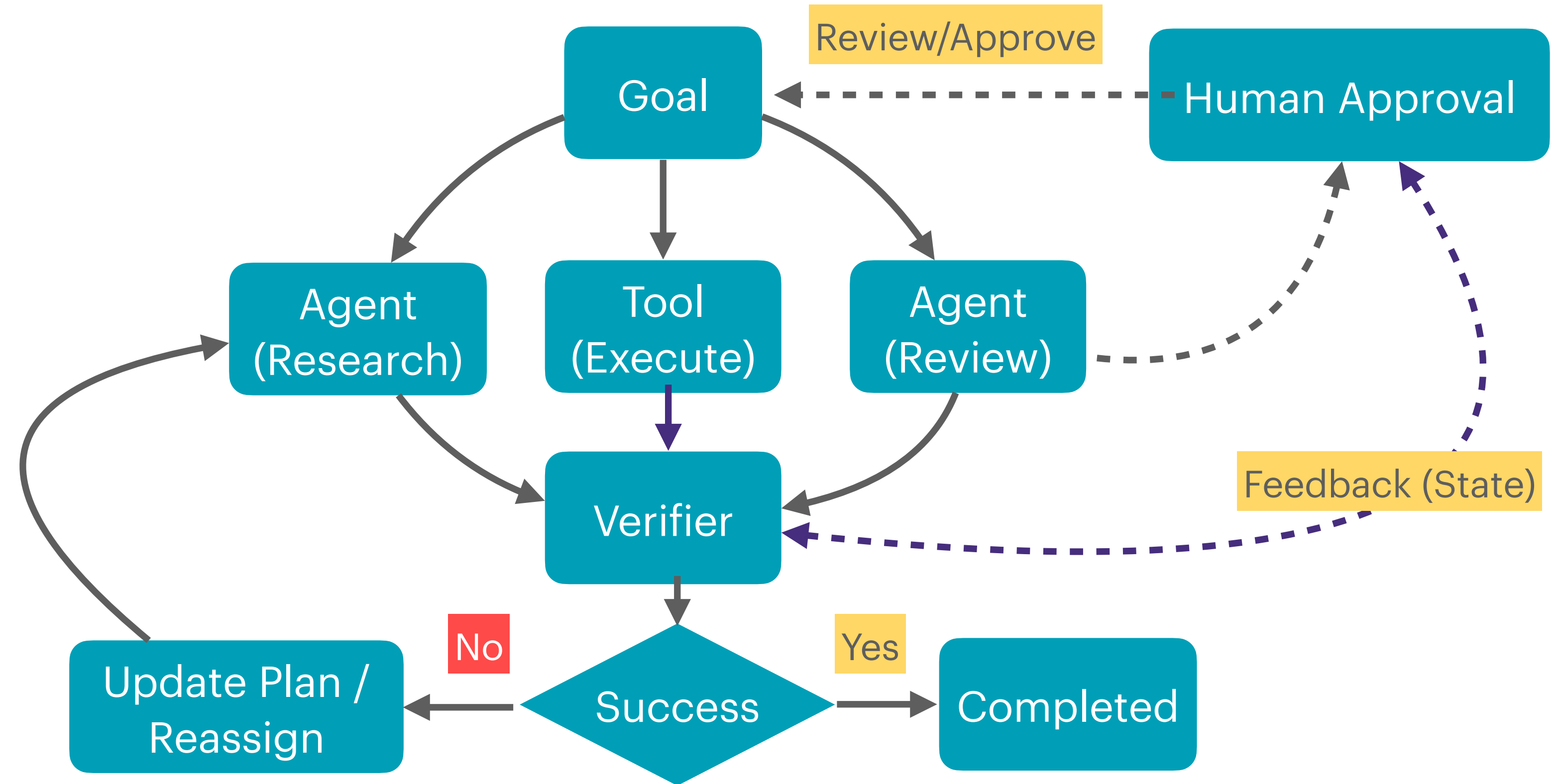


Human: Intent + Constraints >> **Agent:** Reasons + Executes + Reports >> **Human:** Reviews + Judge

Loop ≠ Graph

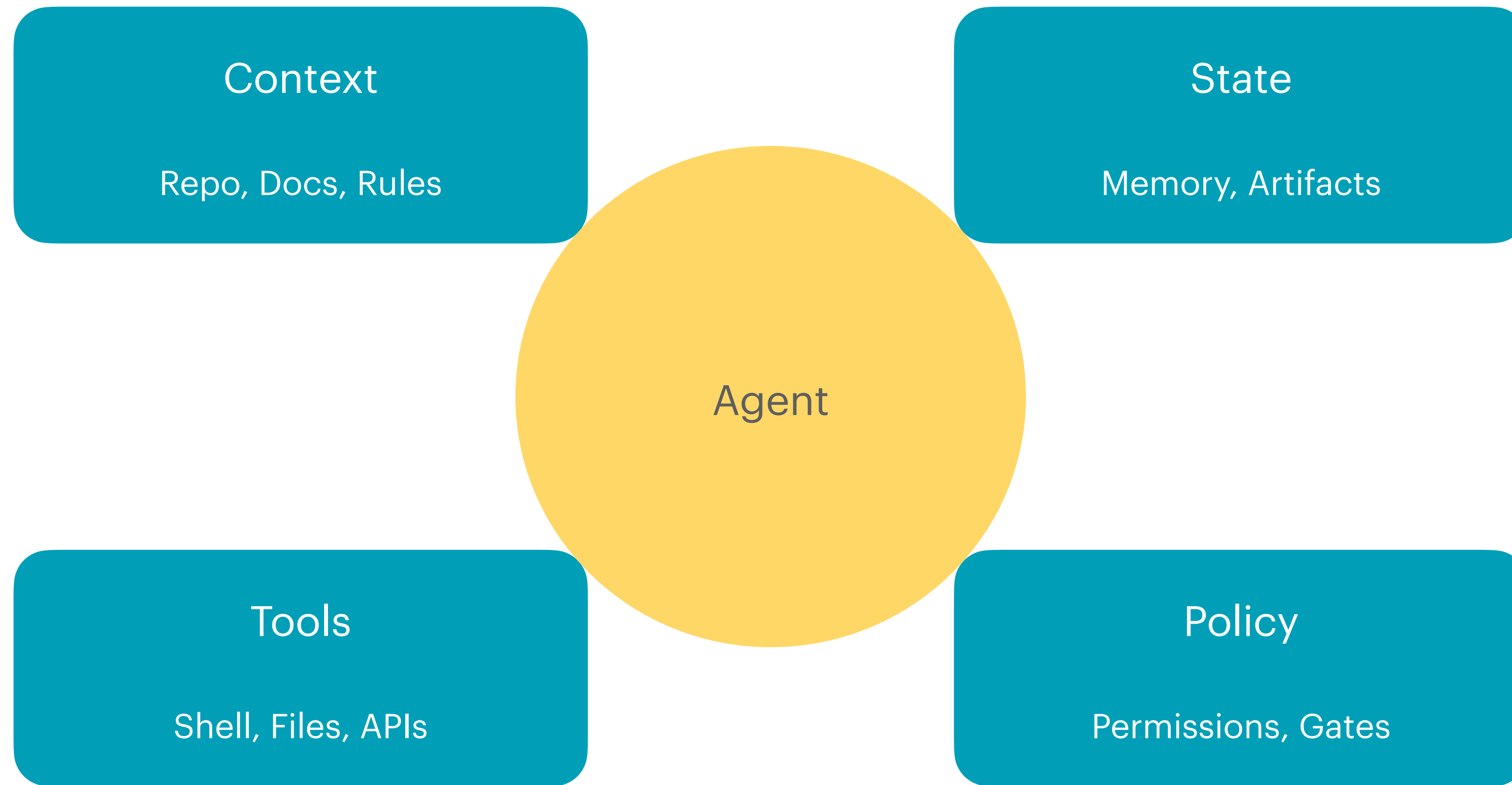


Single agent, Linear sequence,
Repeat until success



Multiple agents, tools, tasks,
Non linear flow, built in verification,
human approval

The Harness: Where does the agent live?



Execution · Observability · Recovery

What Industry Folks Are Saying; Sep 2026



DHH @dhh

Message

It's pencils down, people. Writing code by hand is no longer an economically viable skill for most programmers at most companies. But the future of making software has never been brighter. Don't you dare black pill this beautiful moment!



Andrej Karpathy ... · 30 Apr

This is the the quote I've been citing a lot recently.

kache @yacineMTB · 04 Feb

you can outsource your thinking
but you cannot outsource your
understanding

853 4.6K 46K 2.7M



DHH @dhh · 10h

It takes a while to build the confidence, but there's no future where you're manually reviewing every line of agent code. Not much acceleration in that. You need adversarial agent reviews, you need automated testing, and maybe you spot check. That's it. From prompt to production!

316 359 4.9K 259K



DHH @dhh · 11h

The only programmers who are really in trouble with the competition from artificial intelligence are the ones who insist the world today isn't all that different from the one we lived in last year.

Embrace the paradigm shift or perish.

(This has always been true.)

225 364 4.6K 133K



Gergely Orosz @GergelyOrosz

Message

Grady has lived through several technology changes (microprocessors, the internet, mobile, now AI) and says that when it all changes, some things are the same in software engineering.

He's also saying we're entering another "golden age of software engineering" thanks to AI!



Grady Booch @Grady_Booch · 3d

Replying to @GergelyOrosz

In my career, compilers have always generated better code that I ever could have. I find this to be increasingly so with contemporary AIs.

...

12:50 · 25 Sept 26 · **86.2K** Views

We have often been wrong about the future?

Robert Oppenheimer: After Trinity & Hiroshima, Oppenheimer warned Truman about the profound consequences of atomic weapons - 1945

Nuclear and Space race induced Cold War accidentally built the technological foundations of our world - Sputnik (1957), NASA (1958), Apollo 11 Moon Landing (1969), ARPANET (1960s-70s)

Ken Olsen Founder of DEC: Said: “There is no reason anyone would want a computer in their home.” - 1977

Personal computing became a mass-market phenomenon. The quotation and date are documented by computing-history sources.

We have often been wrong about the future?

Banking & Telecom Industry in India: Strong resistance to computerisation because mechanisation was expected to eliminate jobs - 1980s

India didn't just automate jobs. It created an entirely new economic ecosystem. IT services became one of the most valuable industries.

Geoffrey Hinton: known as Godfather of Modern AI, once said people should stop training radiologists because deep learning would outperform them within 5 years, perhaps 10 - 2016

Years later, the field of Radiology has not disappeared; instead, it faces historic labor shortages, rising demand, and increased employment and salaries. Hinton later acknowledged his error..

Computers didn't simply eliminate work.
They changed what work existed and created
work that was difficult to imagine
beforehand. AI could be no different..

State of (job) market in 2026 /1

- More than 87% had some level of trust in AI's outputs, 31% rise since 2025.
- Nearly 70% of technologists also use AI agents to help them solve problems.
- 80% used AI at least an hour per day; 60% have a favorable attitude towards AI tools.
- Project reqs (85%), technical docs (73%) are most imp sources for context.

<https://survey.stackoverflow.co/2026>

- U.S. job postings requiring AI-literacy skills grew by 70% YoY in 2026
- 1.3 million new AI-enabled jobs emerged globally over 2025-2026; Data annotators, AI engineers, and forward-deployed engineers.
- 75% of global firms agree that people skills, like adaptability, problem-solving, and communication, are even more important in the age of AI.

<https://economicgraph.linkedin.com/research/labor-market-report-2026>

State of (job) market in 2026 /2

- More than 90% of technology professionals now use AI at work, and over 80% believe it has increased their productivity.
- About 59% observe that AI has positively impacted their code quality while 60% of devs use AI to solve a problem / complete a task

<https://dora.dev/research/2025/dora-report/>

- Firms leveraging AI to improve their top line. Since 2022, revenue growth globally per employee in AI-exposed industries like tech/telecom has surged, up 27%.
- Globally, wages are rising more than twice as fast (16.7% vs 7.9%) in industries most exposed to AI, compared to those that are the least exposed.
- 88% of executives say their team or business function plans to increase AI-related budgets in the next 12 months (2025-26) due to agentic AI.

<https://www.pwc.com/us/en/tech-effect/ai-analytics/ai-jobs-barometer.html>

Your fundamentals didn't become obsolete

Computer Science

Algorithms, Data structures
Types, Concurrency
OS, Networks
Databases, Distributed systems
Security, Architecture
User Experience Design
...

Mathematics

Probability, Uncertainty
Logic, Constraints
Discrete structures, Graphs
Statistics, Evaluation
Optimization, Decisions / resources
...

Fundamentals are the vocabulary for engineering AI systems.

What more to learn?

Good General Knowledge, Skills in Multiple Domains, and High Agency

Big-Picture/Systems Thinking, Critical Thinking and Problem Solving

Ask great questions, imagine great products/services, and build/make them

Don't compete with AI at writing code >> Learn to engineer systems with AI.

Lets Build “MediaSync” with AI; OpenCode

- MediaSync is an interactive command-line tool for managing media backup/cleanup
- Uses **adb** to retrieve data from the Android and **rsync** for an incremental backup
- Commands implemented - **backup**, **backup --dry-run**, **cleanup ..**
- Commands to implement - **backup/cleanup --verbose, summary ..**
- <https://github.com/vikrampsingh/vps-mediasync-cli>

Collaborate: Clone repo, Build/Fix Feature, Push, PR -> Review + Merge

Build a “MediaSync” feature using Opencode

References / Further readings

Book: Beyond Vibe Coding by Addy Osmani - <https://beyond.addy.ie/>

Book: Agentic Engineering by Addy Osmani - <https://learning.oreilly.com/library/view/agentic-engineering/0642572392291/>

Paper: The End of Software Engineering by Zhenfeng Cao - <https://arxiv.org/html/2606.05608v1>

Paper: SWE-agent: Agent-Computer Interfaces Enable Automated SWE by John Yang et. al. - <https://arxiv.org/abs/2405.15793>

Paper: SWE-bench: Can Language Models Resolve Real-World Issues? by Carlos E. Jimenez et. al. - <https://arxiv.org/abs/2310.06770>

Report: Stackoverflow - <https://survey.stackoverflow.co/2026>

Report: LinkedIn - <https://economicgraph.linkedin.com/research/labor-market-report-2026>

Report: DORA (Google Cloud) - <https://dora.dev/research/2025/dora-report/>

Report: PwC - <https://www.pwc.com/us/en/tech-effect/ai-analytics/ai-jobs-barometer.html>

Coding Agent: OpenCode - <https://opencode.ai/>

Coding Agent: Open AI Codex - <https://openai.com/codex/>

Coding Agent: Claude Code - <https://claude.com/product/claude-code>

Questions!

Its not the end of engineering, rather end of coding as the scarce skill.

Make a move from writing software to engineering systems that produce software.